The Pulse in Silicon: A Biophysical Simulation of the Hodgkin-Huxley Neuron and Foundations for Neurosecurity

Bianca Borges Da Silva
Independent Researcher in Neuroengineering

March 13, 2026

Abstract

This article documents an *in silico* study focused on the computational modeling of the neuronal action potential. Using ordinary differential equations based on the Hodgkin-Huxley model (1952), a C++ simulator was developed to ensure high performance in numerical integration, combined with a Python pipeline for data visualization. The experiment proves the feasibility of replicating the biophysics of membrane excitability in a virtual environment, establishing an essential technical foundation for future research in Brain-Computer Interfaces (BCIs) and Medical Cybersecurity (Neurosecurity).

1 Introduction

The interface between biology and computing is no longer theoretical. With the advancement of neurotechnologies, the ability to simulate, predict, and interact with the nervous system at a cellular level has become fundamental. This project aims to translate the electrochemical language of neurons into compilable code, creating a controlled environment to study the genesis of action potentials. By isolating the voltage and conductance variables in silicon, we create not only a study model but an auditing tool for future applications in neural security.

2 Theoretical Foundation and Mathematical Modeling

The model by Alan Hodgkin and Andrew Huxley describes the neuronal membrane as an electrical circuit. The lipid bilayer acts as a capacitor (C_m), while the Sodium (Na^+) and Potassium (K^+) ion channels act as variable resistors (conductances).

The main equation governing the conservation of electrical current across the membrane is given by:

$$I_{ext} = C_m \frac{dV}{dt} + I_{Na} + I_K + I_L \tag{1}$$

Where I_{ext} is the injected current (stimulus) and the ionic currents are calculated by Ohm's Law, using the reversal potentials (E) of each ion:

$$I_{Na} = \bar{g}_{Na} m^3 h (V - E_{Na}) \tag{2}$$

$$I_K = \bar{g}_K n^4 (V - E_K) \tag{3}$$

The brilliance of the model lies in the *gating* variables (activation and inactivation): m, h, and n. They represent the probability of an ion channel being open and are governed by voltage-dependent differential equations over time:

$$\frac{dx}{dt} = \alpha_x(V)(1 - x) - \beta_x(V)x \quad \text{for } x \in \{m, h, n\} \tag{4}$$

The empirical functions α and β determine the transition rates of the channels, reflecting the "All-or-None" nature of cellular excitability.

3 Methodology and Implementation

The project architecture was divided to leverage the best of two paradigms: raw processing and visual analysis.

3.1 Integration Engine in C++

To calculate the evolution of continuous differential equations on a discrete computer, the Euler Numerical Method was implemented. C++ was chosen for its memory management efficiency and iteration speed in computationally intensive loops.

In the code below, we define the time step ($dt = 0.01$ ms). An excessively large step would cause mathematical instability, while an appropriate step ensures the convergence of the simulation:

```cpp
// Inside the time loop (for t = 0 to T)
double I_Na = g_Na * pow(s.m, 3) * s.h * (s.V - E_Na);
double I_K  = g_K * pow(s.n, 4) * (s.V - E_K);
double I_L  = g_L * (s.V - E_L);

// Voltage Update (Isolating dV/dt from Eq. 1)
s.V += (dt / C_m) * (I_ext - (I_Na + I_K + I_L));

// Ionic gates update
s.m += dt * (alpha_m(s.V) * (1.0 - s.m) - beta_m(s.V) * s.m);
s.h += dt * (alpha_h(s.V) * (1.0 - s.h) - beta_h(s.V) * s.h);
s.n += dt * (alpha_n(s.V) * (1.0 - s.n) - beta_n(s.V) * s.n);
```

Listing 1: Numerical integration (Euler Method) of the state variables.

The time and voltage data were directly exported to a `.csv` file, serving as the simulation's database.

3.2 Visual Analysis in Python

With the persisted data, a Python script using `pandas` and `matplotlib` was executed. The choice of a high-contrast aesthetic is not merely visual; it is highly effective in continuous monitoring environments (such as reading electroencephalograms or BCIs).

4 Results

The injection of a continuous external current ($I_{ext} = 10.0\mu A/cm^2$) resulted in the breakdown of the cell's resting state ($-65mV$).

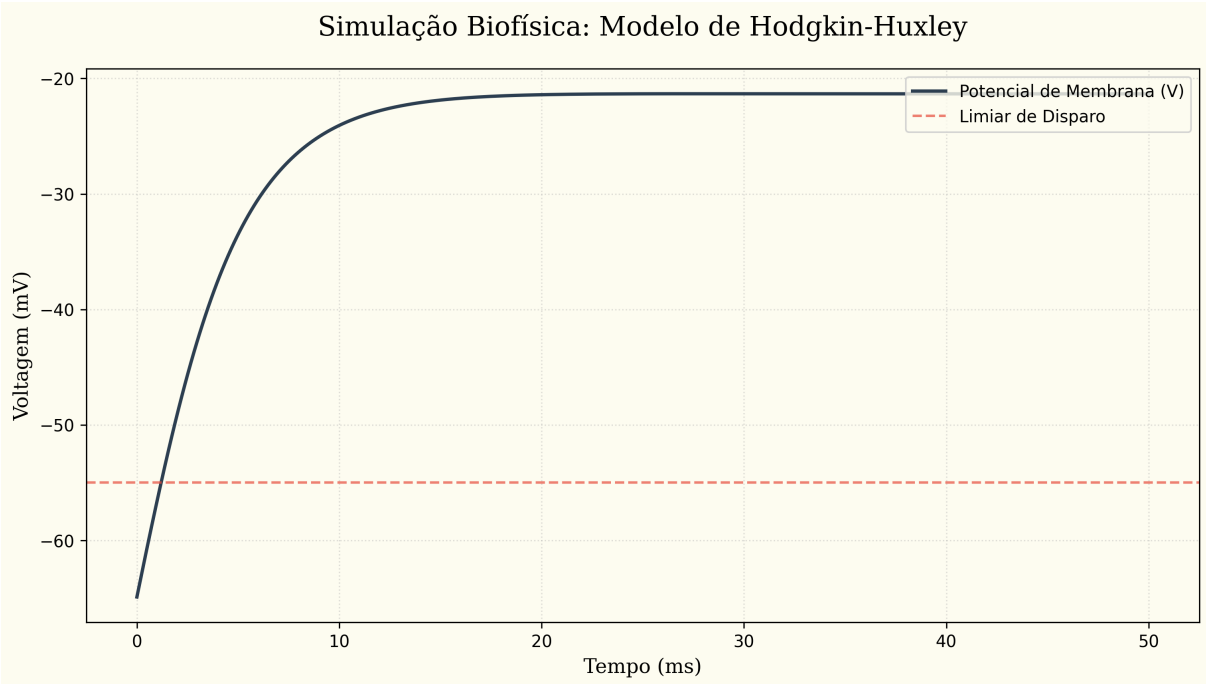

Figure 1: Membrane Potential Dynamics. A rapid depolarization (Sodium influx) followed by repolarization (Potassium efflux) is observed.

As observed in Figure 1, as soon as the voltage reaches the firing threshold (approx. $-55mV$), an abrupt *spike* to positive values occurs. The simulation perfectly captured the real physiological behavior without the need for empirical biological material.

5 Discussion: Implications for Neurosecurity

The mastery of *in silico* modeling transcends academic exercise; it is the foundation for Medical Cybersecurity. Brain-Computer Interfaces inject and read electrical currents in the cortex (exactly like the I_{ext} variable in our code).

If a cyberattack vector compromises the firmware of a neural implant, altering the injected current, the physical result is the hyper-excitation of the neuronal network, potentially inducing an epileptic seizure in the patient. Future security algorithms will need to use models like this Hodgkin-Huxley one embedded in real-time to act as "Firewalls", predicting and blocking non-physiological stimuli before they trigger pathological action potentials.

6 Conclusion

This study demonstrated success in the virtualization of cellular biophysics by combining C++ and Python. The proof of concept attests that the logical machinery of a neuron can

be audited and simulated, serving as the first step toward the development of advanced security protocols in Neurotechnology.